



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: (Fall, Year: 2025), B.Sc. in CSE (Day)*

Chaldal Automation

*Course Title: Software Testing & Quality Assurance Lab
Course Code: CSE 454
Section: 221 D2*

Students Details

Name	ID
Chinmoy Chakma	221002016

*Submission Date: 02-January-2026
Course Teacher's Name: Muntaser Abdul Qader*

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	2
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition	3
1.3.1	Problem Statement	3
1.3.2	Complex Engineering Problem	3
1.4	Design Goals/Objectives	3
1.5	Application	5

Chapter 1

Introduction

1.1 Overview

Focuses on designing and implementing an automated UI testing framework for the Chaldal e-commerce website using Selenium WebDriver and Pytest. The primary objective is to validate the core user flow from login to the checkout slot selection page, ensuring that key functionalities such as authentication, product search, adding items to the shopping bag, and proceeding to checkout remain stable across releases. The automation suite is built using the Page Object Model (POM) architecture to improve maintainability, readability, and scalability by separating page locators and page actions from test logic. The framework makes extensive use of Selenium explicit waits (e.g., visibility checks and clickability checks) to handle dynamic DOM behavior and modern JavaScript-driven UI interactions reliably.

1.2 Motivation

Modern e-commerce platforms are highly dynamic, frequently updated, and customer-impacting. Even small UI changes can break critical business flows such as login, cart actions, or checkout. Manual testing of such flows is time-consuming, inconsistent, and difficult to repeat with the same precision. Therefore, UI automation becomes essential to ensure rapid regression testing, early bug detection, and consistent validation of user journeys. Selenium WebDriver is a widely adopted automation tool for web applications due to its browser automation capabilities, and Pytest is a robust testing framework that offers simple test structuring, fixtures, and reporting support. Additionally, implementing an automation framework with POM reduces the long-term maintenance burden, making it easier to update locators and workflows when the UI changes.

1.3 Problem Definition

1.3.1 Problem Statement

The Chaldal website contains a business-critical flow from user login through product selection and checkout. Due to dynamic UI elements (modal dialogs, overlays, hover-driven buttons, asynchronous content load), manual testing becomes error-prone and slow, while automation attempts may fail due to timing issues (e.g., “Element Not Interactable” errors, overlay blocks, elements with no size/location). The problem addressed in this project is:

How to build a stable, maintainable Selenium + Pytest automation framework that can reliably automate and validate the end-to-end Chaldal user flow from login to checkout slot page, despite UI dynamism and frequent changes.

To address this, the framework applies explicit waits, robust locator strategies, POM-based design, and reusable utility methods to ensure test stability and reduce flaky failures.

1.3.2 Complex Engineering Problem

UI automation testing for modern web platforms is not simply clicking buttons. It involves handling complex issues such as asynchronous rendering, dynamic overlays, element reflow, and multiple modal states. In this project, the complex engineering challenge is ensuring interaction reliability across different DOM states (e.g., login modal transitions and hover-based “Add to Shopping Bag” action). Selenium’s explicit wait mechanisms and structured test design are used as key engineering solutions

1.4 Design Goals/Objectives

The main objectives of this project are:

- **End-to-End Flow Validation** Automate and verify the complete workflow: Login → Search → Add to Bag → Checkout → Slot Page.
- **Maintainable Architecture** Use Page Object Model so locators and actions are separated from test logic, enabling easier updates.
- **Stable Execution (Reduce Flakiness)** Apply explicit waits for visibility/clickability and ensure UI readiness before interaction to avoid timing-related failures.
- **Reusable and Scalable Test Suite** Create reusable base page functions (click, type, wait, scroll, JS click fallback) for future test expansion.
- **Reporting and Debug Support** Generate structured test outputs (e.g., JUnit XML / HTML reports) and logs to help identify failures quickly.

Table 1.1: Summary of the attributes touched by the mentioned projects

Name of the P Attributes	Explain how to address
P1: Depth of knowledge required	Requires deep understanding of Selenium Web-Driver interaction model, DOM synchronization, explicit waits, and Pytest architecture (fixtures, test structure).
P2: Range of conflicting requirements	Balances test stability vs execution speed; also balances robust locators vs minimal dependency on UI changes.
P3: Depth of analysis required	Needs analysis of flaky failures such as “element not interactable,” overlay blocking, and elements without size/location, then designing stable interaction strategies.
P4: Familiarity of issues	Similar issues occur across dynamic web apps (React-based UIs, hover overlays, modal switching); solutions are reusable across projects.
P5: Extent of applicable codes	Uses established best practices: Page Object Model, explicit waits, test modularization, and reporting outputs.
P6: Extent of stakeholder involvement and conflicting requirements	Useful for QA teams, developers, and business stakeholders to validate purchase flows. Conflicting priorities: quick releases vs full regression assurance.
P7: Interdependence	Login → Search → Add-to-cart → Checkout are sequentially dependent; a failure in one step blocks downstream validation. The framework isolates page actions to reduce cascading failures.

1.5 Application

The real-world application of this project is directly aligned with industry needs for regression testing in e-commerce platforms. The automation framework can be used by QA engineers and developers to validate critical customer journeys before production deployment. It reduces release risk by quickly detecting broken UI elements or workflow failures. In addition, the framework can be integrated into CI/CD pipelines where tests run automatically on every build, ensuring continuous validation of key purchase flows. This is particularly important for Chaldal-like platforms where broken checkout flows can cause revenue loss and customer dissatisfaction. Furthermore, the same framework can be extended to support additional test scenarios such as payment validation, coupon application, address selection, and delivery slot confirmation.